Nimbus P4: Foundation and Cloud Infrastructure Summary

Live Deployment: AWS Account `930124786284`, Region `ap-south-1`, Service: `nimbus-ds252-service`

Executive Overview of P4 Deliverables

Person 4 delivered the production cloud infrastructure and deployment foundation for Project Nimbus, transitioning the system from local simulation to a live, monitored AWS environment. The implementation strictly adheres to a vanilla, lightweight stack: standard library Python HTTP server (`server.py`), standard AWS SDK (`boto3`), and vanilla browser assets (HTML, CSS, JavaScript) without external application frameworks.

- **Live Cloud Service:** Automated Terraform provisioning and containerized deployment on AWS ECS Fargate, served via Amazon API Gateway HTTP API (`qrgdidp9te.execute-api.ap-south-1.amazonaws.com`).
- **Requester Web Application:** Lightweight hospital logistics portal (`static/requester.html`, `static/requester.js`) at root route (`/`) providing natural language item dispatch, clarification response intake, and real-time task status polling.
- **Operator Command Console:** Operational command dashboard (`static/operator.html`, `static/operator.js`) at `/operator` featuring live fleet telemetry, link fault monitoring, active task tracking, and runtime controls.
- **Operator Inventory Controls:** Direct inventory counters (`data/inventory.json`, `/inventory`) in the console with exactly two buttons: "Set stock" and "Reset stock". When tasks are leased, stock is automatically decremented. Memory can also be reset via `POST /reset-memory`.
- **Active Pack Switch to `hospital_sim6`:** Migrated the default domain configuration to `packs/hospital_sim6` (ID: `hospital_sim6@0.1.0:88d32afe`) across `packs/DEFAULT`, `Dockerfile`, and ECS task definition. This configures the six-robot Gazebo world (`R1`–`R3` in `east-wing`, `R4`–`R6` in `west-wing`) with P1 Gazebo map topology.

Requester Application and Cloud Processing Path

Input Schema (`POST /requests`)

Hospital requests are submitted as JSON payloads to `POST /requests`:

- `text` (`string`, required): Natural language request, e.g., `"Deliver a wheelchair to ward 3, routine."` or `"Need an infusion pump at 2B-12, urgent."`
- `message_id` (`string`, required): Unique client UUID/token used for strict idempotency deduplication (PRD FR-8).

Request Processing Pipeline

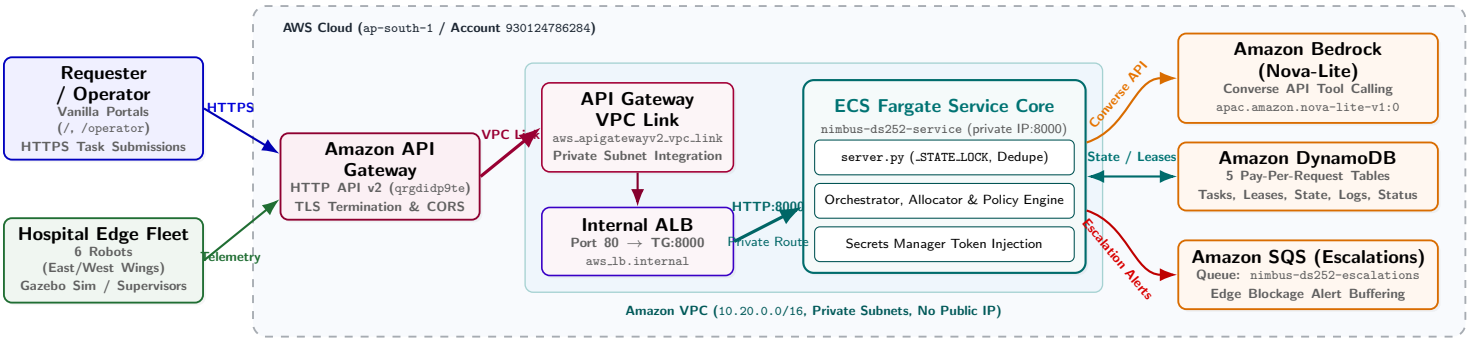
1. **Deduplication (`server.py:510`):** Incoming `message_id` is evaluated under thread lock `_STATE_LOCK`. If previously recorded in store, the call immediately returns HTTP 200 with {`"duplicate": true`} without re-execution.
2. **Context Assembly (`cloud/orchestrator.py:207`):** Orchestrator calls `build_context()` to synthesize map topology nodes, active closures, fleet positions, battery states, stock counts, and prior task history.
3. **Intent Parsing (`cloud/agent.py`, `cloud/llm.py`):** In `C1` mode, requests invoke Amazon Bedrock (`BedrockLLM`) with Nova-Lite to produce structured tool calls (`submit_task_spec` or `ask_clarification`). If Bedrock is down or unreachable, the circuit breaker instantly diverts execution to the deterministic fallback (`RuleBasedLLM`) with reduced autonomy notification.
4. **Domain Rule and Policy Check (`cloud/policy.py`, `cloud/allocator.py`):** Validates item constraints (e.g., paediatric equipment requirements under `SOP-PAED-01`), area closures, and isolation ward restrictions (`SOP-ISO-02`).
5. **Cluster Allocation & Routing (`cloud/allocator.py`):** Identifies candidate robot clusters (`east-wing` vs. `west-wing`), verifies equipment stock, computes shortest path on the Gazebo topology graph, and selects available robots.
6. **Lease Synthesis (`cloud/leasebuilder.py`):** Constructs an authority lease with strict monotonic versioning, allowed/forbidden zones, robot sequences (`go_to`, `wait`, `photo`, `report`), speed limits, and escalation triggers.
7. **Inventory Decrement and State Persistence (`server.py:804`, `cloud/aws_dynamo.py`):** Decrements local/persisted stock for the requested item and writes tasks, leases, and decision rows to DynamoDB tables.

Output Schema

- **Synchronous API Response (`POST /requests`):** Returns HTTP 201 Created:
  - {`"duplicate": false, "task_id": "task-XXXXXX", "status": "leased" | "needs_clarification", ...`}
- **Authority Lease Contract (`contracts/lease.schema.json`):** If leased, the authoritative contract provides:
  - `lease_id` (`string`): Globally unique identifier (e.g., `"lease-task-000027-v1"`).
  - `version` (`integer`): Monotonic version number; newer versions completely supersede older versions.
  - `priority` (`"routine"` or `"urgent"`): Task priority class.
  - `deadline_ts` and `expiry_ts` (`ISO-8601`): Task deadline and hard expiry timestamps.
  - `allowed_zones` and `forbidden_zones` (`array[string]`): Geofenced graph nodes.
  - `speed_limit` (`number`): Maximum speed limit in meters per second (1.0 m/s in simulation).
  - `assigned_robots` (`array[string]`): Designated robots from the pack fleet (e.g., `["R6"]`).
  - `task_sequence` (`array[object]`): Sequential robot commands with parameters and target nodes.
  - `cluster_id` (`"east-wing"` or `"west-wing"`): Assigned responsible edge cluster.
  - `when_to_escalate` (`array[string]`): Guard conditions trigger escalation (e.g., `"door_closed"`).

Live AWS Infrastructure Architecture

The cloud infrastructure is provisioned through modular Terraform configurations located in `infra/terraform/` and operates in AWS Account `930124786284`, Region `ap-south-1`.



- **ECS Fargate (`nimbus-ds252-cluster` / `nimbus-ds252-service`):** Runs container `nimbus-app` on `linux/amd64` with 512 CPU units and 1024 MB RAM, deployed in private subnets **without** public IP assignment (`assign_public_ip = false`). Tasks receive traffic exclusively via target group attachments from the internal ALB, with ingress restricted to the ALB security group on port 8000. Application secrets (`NIMBUS_*_TOKEN`) are injected dynamically into the container environment from AWS Secrets Manager.
- **Amazon API Gateway (`qrgdidp9te`) & Private VPC Link:** HTTP API v2 providing public HTTPS termination. Integrations use a private `VPC_LINK` (`aws_apigatewayv2_vpc_link.main`) routing traffic into an internal Application Load Balancer (`aws_lb.internal`) listening on port 80 across private subnets, completely eliminating the need for public IPs on ECS tasks.
- **Deployment Identification & Local Safety:** Live AWS identifiers (`930124786284`, `qrgdidp9te`) document the evaluated cloud target. Local execution and test runners default strictly to local cloud (`NIMBUS_LOCAL_CLOUD=1`) to avoid inadvertent access to production resources.
- **Amazon DynamoDB (Pay-Per-Request Tables):**
  - `nimbus-ds252-tasks`: Stores task metadata, status, request text, and current lease state.
  - `nimbus-ds252-leases`: Stores monotonic authority leases keyed by `lease_id`.
  - `nimbus-ds252-state`: Holds place inventory, dynamic corridor closures, and TTL-expiring items.
  - `nimbus-ds252-decision_log`: Immutable timeline of decisions and raw event inputs/outputs for auditability.
  - `nimbus-ds252-robot_status`: Records real-time robot check-in status, battery levels, and telemetry.
- **Amazon SQS (`nimbus-ds252-escalations`):** Buffers critical edge escalation events and operator alerts when automated resolution is blocked.
- **Amazon Bedrock (`apac.amazon.nova-lite-v1:0`):** Serverless foundation model accessed via the Converse API for semantic parsing and structured parameter extraction.

System Guardrails and Resilience Mechanisms

- **LLM Circuit Breaker (`cloud/llm.py:227`):** A three-state circuit breaker (`CircuitBreakerLLM`) tracks model failures. Upon 3 consecutive exceptions or timeouts, the circuit transitions from `closed` to `open`. While open, all calls fail fast without network delay, immediately activating deterministic regex-based rules (`RuleBasedLLM`) with a `Reduced autonomy` notice. After 30 seconds of cooldown, the breaker enters `half_open` to test model recovery.
- **JSON Schema Validation (`contracts.py`, `contracts/`):** Outgoing leases, incoming escalations, and robot sub-tasks are validated against formal JSON Schema Draft 2020-12 specifications. Any lease violating constraints throws an immediate error preventing invalid dispatch.
- **State Locking and Concurrency Control (`server.py`):** An internal reentrant thread lock (`_STATE_LOCK`) wraps critical sections across deduplication, task creation, stock updates, memory resets, and queue draining, eliminating race conditions.
- **Monotonic Lease Versioning (PRD FR-8):** Every lease increment advances a monotonic version counter. Edge robots reject any lease version equal to or lower than their active lease, safeguarding against out-of-order execution.
- **Hospital SOP Rules and Isolation Boundaries:** Policies like `SOP-PAED-01` mandate paediatric equipment validation. Dynamically declared isolation zones or corridor closures are maintained in `PolicyBook` and immediately restrict route planning.
- **Priority Queue Draining (`cloud/scheduler.py`):** When inventory or route availability changes (e.g., via "Set stock" or corridor reopening), the scheduler automatically drains queued tasks in strict priority and deadline order.

Grounding References

All information in this document is grounded in: `server.py`, `Dockerfile`, `infra/terraform/main.tf`, `cloud/orchestrator.py`, `cloud/llm.py`, `cloud/durable.py`, `cloud/policy.py`, `cloud/allocator.py`,

cloud/leasebuilder.py, cloud/aws_dynamo.py, contracts/lease.schema.json, packs/hospital_sim6/, and static/operator.html.